SIR SYED UNIVERSITY OF ENGINEERING & TECHNOLOGY

COMPUTER ENGINEERING DEPARTMENT

Software Requirement Specification

LogAI | AI-Powered Log Monitoring Dashboard

B.S. in Computer Engineering, Batch 2022F

Internal Advisor: _____

Project Team

Name	Roll No	Section	Role	Contact
Khalid Hussain	2022F-BCE-130	B	Group Leader	03322425697
Nauman Khalid	2022F-BCE-147	A	Member	03295658357
Ahmed Sartaj	2022F-BCE-212	A	Member	03303092753
Nauroz Saleem	2022F-BCE-150	A	Member	03032809169

Karachi, Pakistan

May 29, 2026

Contents

List of Figures

1.1 Introduction

LogAI is a web-based observability platform designed to centralize log ingestion, real-time monitoring, anomaly detection, alerting, and analytics for modern software systems.

The system combines a React frontend, a FastAPI backend, a dedicated Node.js auth service, Redis-based streaming, Elasticsearch-based search and analytics, PostgreSQL-based user metadata, and containerized deployment assets.

This SRS documents the current product requirements, interfaces, architecture, and design expectations for the final year project build. Performance values in this report are acceptance targets and should be validated against the live stack during the final verification run.

1.1.1 Product Purpose

The purpose of LogAI is to help operations teams, developers, and system administrators collect, observe, and analyze application logs from multiple sources in one place.

The system is intended to shorten debugging time, improve operational awareness, surface anomalies quickly, and provide a cleaner workflow for log search, live monitoring, and outbound alerting.

1.1.2 Product Scope

LogAI supports secure user authentication, monitored server registration, API-key-based log ingest, Fluentd-based collection, log parsing, anomaly scoring, real-time streaming to the dashboard, search and filtering, analytics, alert integrations, and a lightweight chat assistant that explains log context.

The current build also includes starter CI/CD, Docker Compose deployment assets, a reusable testing layer, and an Isolation Forest based anomaly engine with a statistical fallback path.

> *Performance numbers and final screenshots should be updated after the live deployment test pass.*

1.1.3 System Diagram and Description

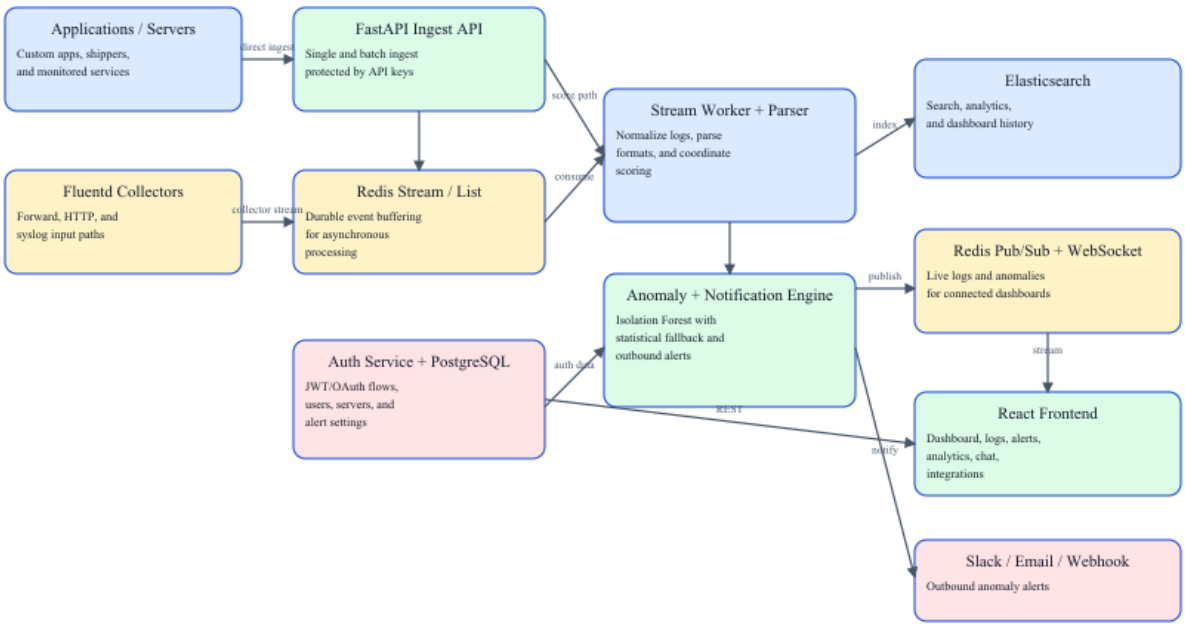

Figure 1.1: LogAI System Diagram

- Applications and servers submit logs either directly to the FastAPI ingest API or through Fluentd collectors.

- Redis Streams and Redis Lists decouple ingestion from processing so the stream worker can parse, score, and route logs reliably.

- The anomaly engine evaluates each log entry, then indexed logs are stored in Elasticsearch while live updates are published to Redis Pub/Sub.

- The React dashboard consumes historical data through REST APIs and live updates through the WebSocket endpoint.

- When anomalies exceed a configured threshold, outbound notifications can be dispatched through Slack, email, and generic webhooks.

1.1.4 Project Stakeholders

Stakeholder	Role / Interest
DevOps Engineers / SREs	Primary users who monitor infrastructure health, incidents, and log anomalies.
Software Developers	Investigate errors, search logs, and use analytics and chat assistance during debugging.
System Administrators	Maintain the deployment, configure infrastructure, rotate keys, and manage environments.
Project Team	Responsible for requirements, implementation, documentation, and future improvement of the system
Project Advisor / Academic Evaluators	Review system scope, architecture, correctness, testing, and deliverables.
Cloud / Hosting Providers	Provide the runtime resources used by the databases, backend services, and frontend deployment.
Alert Delivery Services	Slack workspaces, SMTP providers, and webhook receivers that deliver anomaly notifications.

1.1.5 Project WBS and Gantt Chart

Work Breakdown Structure

WBS	Work Package	Description
1.0	Requirement analysis and proposal alignment	Clarify scope, required features, and evaluation milestones.
2.0	Backend platform	Authentication, server management, ingest routes, search, analytics, and chat APIs.
3.0	Frontend dashboard	Connected user interface for dashboard, logs, alerts, analytics, servers, chat, and integrations
4.0	Real-time pipeline	Redis streams, stream worker, WebSocket live updates, and anomaly event propagation.
5.0	Deployment structure	Docker Compose stack, CI/CD starter, and workspace cleanup.
6.0	Alerting and AI/ML	Slack/email/webhook integrations and Isolation Forest anomaly scoring.
7.0	Testing and documentation	Smoke tests, load tests, architecture notes, run guides, and report artifacts.

Phase-wise Gantt overview

Task	P1	P2	P3	P4	P5	P6	P7
Requirement analysis	**Active**						
Backend core implementation	**Active**	**Active**					
Frontend integration and UX		**Active**	**Active**	**Active**			
Real-time pipeline		**Active**	**Active**				
Deployment and repo structure			**Active**	**Active**			
Alerting and integrations					**Active**		
AI/ML upgrade						**Active**	
Hardening, testing, documentation							**Active**

1.1.6 Assumptions and Constraints

- The deployment environment provides Docker Desktop or an equivalent container runtime for the local stack.

- Node.js 18+ and Python 3.11+ are available when frontend development and local testing scripts must be run outside containers.

- SMTP, Slack, webhook, and OAuth behavior depends on valid third-party credentials being supplied through environment variables.

- The current deployment path is Docker Compose for local and demo execution.

- The current chat assistant is retrieval and rule based; it should not be described as a general-purpose foundation model.

- Final performance numbers and screenshots depend on a live system test pass and are not fully measurable from static code inspection alone.

1.2 Project Functional Requirements

1.2.1 Project Functional Features

ID	Requirement	Description	Priority
FR-01	User registration and login	The system shall support local email/password signup, login, logout, token refresh, and profile retrieval.	High
FR-02	OAuth callback support	The system shall support Google and GitHub OAuth redirection through the auth service.	Medium
FR-03	Server management	The system shall allow an authenticated user to create, list, soft-delete, and rotate API keys for monitored servers.	High
FR-04	Single log ingest	The system shall accept a single log entry through an API-key-protected ingest endpoint.	High
FR-05	Batch log ingest	The system shall accept up to 1000 log entries in one batch request.	High
FR-06	Collector-based ingest	The system shall support Fluentd and Redis-based pipeline ingest in addition to direct API ingest.	High
FR-07	Log parsing and normalization	The system shall parse common raw log formats and normalize them into a shared searchable structure.	High
FR-08	Real-time streaming	The system shall push new logs and anomalies to connected dashboard clients through WebSocket updates.	High
FR-09	Search and filtering	The system shall let users query logs by server, severity, anomaly flag, text search, and time range.	High
FR-10	Dashboard metrics	The system shall present 24-hour overview metrics, severity breakdowns, hourly activity, and per-server health summ	High
FR-11	Analytics views	The system shall provide detailed server metrics, anomaly counts, severity charts, and trend data.	High
FR-12	Alert management	The system shall display anomaly events and support live alert updates in the frontend.	High
FR-13	Outbound integrations	The system shall allow users to configure Slack, email, and webhook channels and send test notifications.	High
FR-14	AI-assisted chat	The system shall let users ask questions about their logs and receive Elasticsearch-backed contextual responses derive	Medium
FR-15	Deployment assets	The system shall include Docker-based local deployment assets for repeatable setup and demonstration.	Medium
FR-16	Testing support	The system shall include smoke-test and ingest load-test utilities for final verification.	Medium

1.2.1.1 Chat Assistant Details

- The current chat feature is not a hosted large language model integration in the deployed build.

- It works by querying Elasticsearch for recent logs relevant to the user's message, then composing a contextual response through rule-based response logic in the backend.

- Its data source is therefore the user's own indexed log history, filtered by owned servers and recent time windows.

- This makes the current build deterministic, lightweight, and fully local to the project stack, while still giving useful debugging assistance.

1.2.2 Operating Environment

Environment Item	Current Build Expectation
Client browser	Chrome, Edge, or Firefox on Windows, Linux, or macOS
Frontend runtime	Node.js 18+ during development, static frontend served by NGINX in deployment
Backend runtime	Python 3.11+ for FastAPI backend and stream worker
Auth runtime	Node.js service for OAuth and callback handling
Primary databases	PostgreSQL 16, Elasticsearch 8.x, Redis 7
Collection layer	Fluentd with forward, HTTP, and syslog inputs
Container environment	Docker Desktop / Docker Compose for local stack execution
Network prerequisites	HTTP/HTTPS access plus database, Redis, and collector ports where applicable

1.2.3 UX Prototype

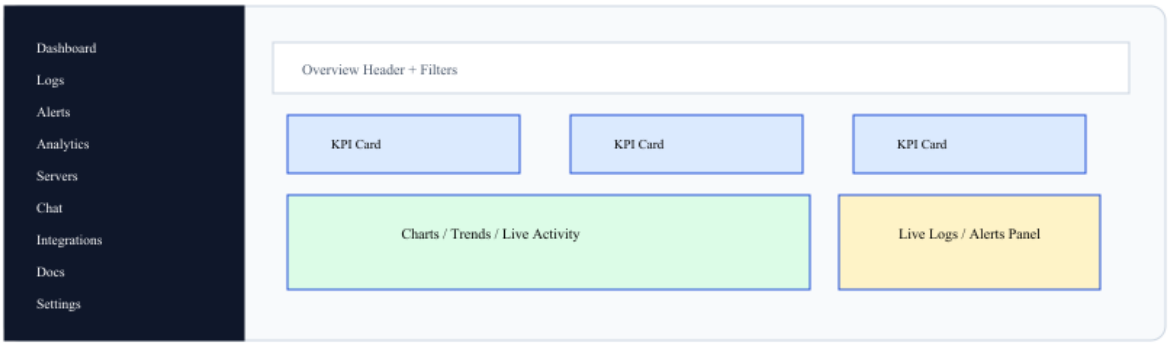

Figure 1.2: UI/UX Prototype Overview

Authentication: Login and signup flow with local auth and OAuth callback support.

Dashboard: Overview cards, live logs, anomaly preview, and per-server summary.

Logs: Search, filters, pagination, and anomaly-only exploration.

Alerts: Live anomaly view with server scoping and feed behavior.

Analytics: Trend charts, severity breakdowns, and anomaly metrics.

Servers: Create/list servers, view API keys, and manage monitored systems.

Integrations: Configure Slack/email/webhook delivery and send test alerts.

Chat: Ask log-related questions and receive contextual responses.

1.3 Project Non-Functional Requirements

1.3.1 Performance Evaluation

The following values are acceptance targets pending the final live verification run.

Metric	Acceptance Target
Health endpoint response	< 1 second under normal local deployment load
Single ingest request	< 500 ms average under light load
Batch ingest limit	Up to 1000 log entries per request
Dashboard overview query	< 3 seconds for the 24-hour aggregate view
Log search query	< 2 seconds for common filtered queries with paginated output
WebSocket live update delivery	Visible in the frontend within 2 seconds after processing
Notification dispatch	< 10 seconds for configured Slack/email/webhook channels
Load-test baseline	Support a local benchmark run of at least 200-300 ingest requests using the included test script

1.3.2 Safety Requirements

- The system shall never perform destructive remediation actions automatically on monitored infrastructure.

- The system shall preserve log evidence and anomaly metadata even when live streaming is temporarily unavailable.

- The system shall fail gracefully by returning informative errors instead of silently dropping authenticated user actions.

- The system shall continue using the statistical anomaly fallback when the ML model is not trained or unavailable.

1.3.3 Security Requirements

- JWT access and refresh tokens shall be used for authenticated user APIs.

- Passwords shall be stored as bcrypt hashes and never persisted in plain text.

- API-key-based ingest shall be isolated per monitored server.

- User access to servers, metrics, logs, and integrations shall be scoped to owned resources only.

- Rate limiting shall be applied to sensitive API flows such as ingest traffic.

- Secrets such as JWT keys, OAuth credentials, SMTP credentials, and webhook URLs shall be supplied through environment variables or secret stores.

- Production deployment should enable HTTPS/TLS at the reverse-proxy layer.

1.3.4 Business Perspective / Requirements

- LogAI addresses a practical observability need for student projects, internal systems, and small to medium operational teams that need centralized log visibility without a large commercial stack.

- The product reduces manual debugging effort by combining live streams, stored history, analytics, anomaly scoring, and outbound alerting in one workflow.

- The platform is intentionally modular so future work can add richer ML, hosted deployment hardening, or organization-level roles without redesigning the whole system.

1.3.5 Licensing Requirements

- The current build depends on open-source frameworks and libraries such as React, FastAPI, SQLAlchemy, Redis clients, Elasticsearch clients, and report/test tooling.

- Any public distribution of the project should include a top-level project license and third-party attribution review before release.

- External services such as Slack, SMTP providers, and cloud infrastructure remain subject to their own service terms and licenses.

1.3.6 Legal Copyrights and Notices

- Organizations deploying LogAI remain responsible for the legality of collected log data, retention policies, and privacy obligations.

- The system may process sensitive operational or personal data if applications emit it into logs; therefore, deployers should implement masking, retention limits, and access controls appropriate to their context.

- Evidence, alerts, and exported diagnostics should be handled according to local security policies and any applicable data-protection regulations.

1.3.7 Verification and Testing Strategy

ID	Validation Area	Expected Check	Method
TS-01	Health verification	Validate backend health and auth health endpoints before feature testing.	Automated smoke test
TS-02	Authentication flow	Verify signup, login, token refresh, logout, and OAuth callback behavior.	Manual bug bash
TS-03	Server lifecycle	Create servers, list them, and rotate keys without cross-user access leakage.	Manual + API check
TS-04	Ingest validation	Send single and batch logs with valid API keys and confirm persistence.	Manual + smoke test
TS-05	WebSocket validation	Confirm live logs and anomaly events reach the dashboard in real time.	Manual live test
TS-06	Analytics and alerts	Validate alert feed, metrics pages, and anomaly counters against ingested data.	Manual live test
TS-07	Integration delivery	Save Slack/email/webhook configuration and send a synthetic test notification.	Manual live test
TS-08	Anomaly-model validation	Warm the model with at least 100 logs and compare fallback vs ML-active behavior.	Manual live test
TS-09	Load testing	Run the ingest load test script and record throughput plus latency metrics.	Automated load test

1.3.8 Risks and Mitigation

Risk ID	Risk	Impact	Mitigation
R-01	Backend or database service failure	Dashboard pages may fail to load or ingest may stop.	Use health checks, container restarts, and service-level logs to detect and rec
R-02	High ingest load	Queue build-up, slower indexing, or delayed Redis buffering.	Use Redis buffering, batch indexing, load testing, and future horizontal scali
R-03	Notification channel misconfiguration	Important anomaly alerts may not be delivered.	Provide test-send flows, readiness indicators, and per-channel configuration
R-04	Model false positives or false negatives	Users may miss incidents or get noisy alerts.	Keep statistical fallback, tune thresholds, and document model warm-up beh
R-05	Invalid or noisy log payloads	Parsed data quality and analytics accuracy may degrade.	Normalize formats, support raw preservation, and store metadata for later ins
R-06	Secret leakage or weak configuration	Unauthorized access to APIs or third-party integrations.	Use environment variables, secret stores, API key rotation, and JWT-based a

1.4 Software Design Methodology

The system follows a modular, service-oriented design with clear separation between user-facing frontend flows, API orchestration, asynchronous stream processing, storage layers, and alert delivery.

Development is aligned with an iterative and incremental approach: core backend services, connected frontend flows, live streaming, deployment structure, alerting, AI/ML upgrade, and final hardening were introduced in phases.

The backend uses REST for historical and administrative operations, WebSocket for live updates, Redis for event movement and rate limiting, Elasticsearch for search and analytics, and PostgreSQL for user-owned metadata.

1.4.1 Data Dictionary / Data Set

Relational entities

Field	Type	Description
users.id	UUID	Primary key for each user.
users.name	String(255)	Display name of the user.
users.email	String(255)	Unique login and contact identity.
users.hashed_password	String(255)	Bcrypt-hashed password for local auth.
users.picture	String(1024)	Optional avatar URL.
users.auth_provider	Enum	Authentication source: local, google, or github.
users.oauth_id	String(255)	Optional OAuth provider identifier.
users.is_active	Boolean	Soft-active flag for the account.
servers.id	UUID	Primary key for each monitored server/application.
servers.name	String(255)	Display name of the monitored server.
servers.description	Text	Optional descriptive notes.
servers.api_key	String(255)	Unique API key used for log ingestion.
servers.owner_id	UUID	Foreign key to the owning user.
servers.is_active	Boolean	Soft-active flag for the server.
alert_integrations.owner_id	UUID	Unique foreign key linking one integration profile to one user.
alert_integrations.slack_enabled	Boolean	Enables Slack notifications.
alert_integrations.slack_webhook_url	String(1024)	Slack Incoming Webhook URL.
alert_integrations.email_enabled	Boolean	Enables email notifications.
alert_integrations.email_recipients	Text	Comma-separated email recipients.
alert_integrations.webhook_enabled	Boolean	Enables generic webhook delivery.
alert_integrations.webhook_url	String(1024)	Webhook endpoint URL.
alert_integrations.minimum_anomaly_score	Float	Threshold required before outbound alerting.

Log index dataset

Field	Type	Description
logai-logs.id	keyword	Unique log document identifier.
logai-logs.server_id	keyword	Owning server identifier.
logai-logs.server_name	keyword	Human-readable server name.

Field	Type	Description
logai-logs.level	keyword	Normalized severity level.
logai-logs.message	text + keyword	Primary searchable log message.
logai-logs.source	keyword	Logger or collector source.
logai-logs.host	keyword	Origin host name.
logai-logs.service	keyword	Application service or subsystem.
logai-logs.environment	keyword	Environment tag such as dev/staging/prod.
logai-logs.meta	object	Dynamic structured metadata payload.
logai-logs.raw	text	Original raw message when preserved.
logai-logs.anomaly	boolean	Boolean anomaly classification.
logai-logs.anomaly_score	float	0..1 anomaly score.
logai-logs.timestamp	date	Primary event time in epoch millis.
logai-logs.ingested_at	date	Time the system accepted the record.
logai-logs.processed_by	keyword	Pipeline source such as api-direct, stream, or fluentd.

1.4.2 Design Models [along with descriptions]

Model	Purpose
System architecture model	Describes the high-level components and data paths from sources to dashboards and alert channels.
Use case model	Shows how authenticated users, monitored applications, and notification channels interact with the syste
Entity relationship model	Defines the persistent ownership relationships between users, servers, alert integrations, and log docume
Sequence model	Explains the order of calls during a log-ingest-to-dashboard event.

1.4.2.1 Use Case Diagram

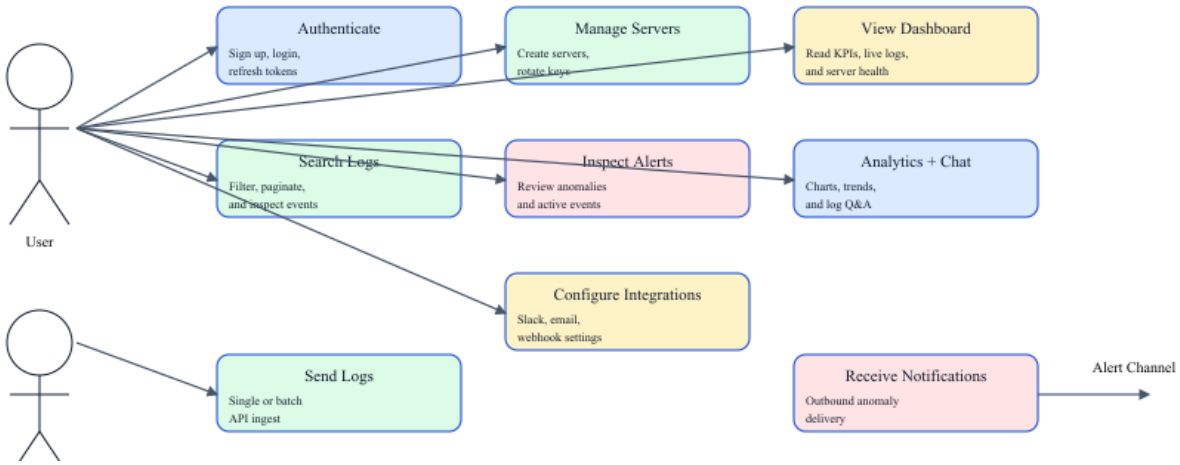

Figure 1.3: Use Case Diagram

1.5 External Interface Requirements

API and service interfaces

Interface	Protocol	Direction	Description
POST /api/v1/auth/signup	HTTPS	Client -> FastAPI	Register a local user account and receive access/refresh tokens.
POST /api/v1/auth/login	HTTPS	Client -> FastAPI	Authenticate a local user and receive tokens.
POST /api/v1/auth/refresh	HTTPS	Client -> FastAPI	Issue a new access token using a refresh token.
GET /api/v1/auth/me	HTTPS	Client -> FastAPI	Return the authenticated user profile.
GET /api/v1/servers	HTTPS	Client -> FastAPI	List owned servers with 24-hour statistics.
POST /api/v1/servers	HTTPS	Client -> FastAPI	Create a monitored server and API key.
GET /api/v1/servers/{id}/metrics	HTTPS	Client -> FastAPI	Fetch detailed server metrics.
GET /api/v1/servers/dashboard/overview	HTTPS	Client -> FastAPI	Fetch dashboard-wide overview data.
POST /api/v1/ingest	HTTPS + x-api-key	Application -> FastAPI	Submit one log entry.
POST /api/v1/ingest/batch	HTTPS + x-api-key	Application -> FastAPI	Submit a batch of logs.
GET /api/v1/logs	HTTPS	Client -> FastAPI	Search and filter indexed logs.
POST /api/v1/chat	HTTPS	Client -> FastAPI	Ask a contextual question about current logs.
GET /api/v1/integrations/alerts	HTTPS	Client -> FastAPI	Load alert integration settings.
PUT /api/v1/integrations/alerts	HTTPS	Client -> FastAPI	Update alert delivery settings.
POST /api/v1/integrations/alerts/test	HTTPS	Client -> FastAPI	Send a synthetic test alert.
WS /ws?token=...&server_id=...	WebSocket	Client <-> FastAPI	Receive live log and anomaly events and switch subscriptions.
GET /api/auth/google, /api/auth/github	HTTPS	Client -> Auth Service	Start OAuth authentication flows.
GET /api/auth/health	HTTPS	Client -> Auth Service	Health endpoint for auth service validation.

Communication interfaces

Communication Path	Technology
Frontend to backend	HTTPS REST + WebSocket
Backend to PostgreSQL	Async PostgreSQL driver
Backend / worker to Redis	Redis streams, lists, pub/sub, and sorted sets
Backend / worker to Elasticsearch	HTTP API via async client
Fluentd to Redis	Collector pipeline
Alert delivery	Slack webhook, SMTP, generic outbound webhook

Recommended manual bug-bash coverage

- Authentication and token refresh
- OAuth callback flow
- Server creation and API key rotation
- Single and batch ingest
- Live dashboard WebSocket stream
- Logs filtering and pagination
- Alerts feed behavior

- Analytics values and charts
- Integrations save and test-send flow
- Chat response quality for recent logs

1.6 Entity Relationship Diagram

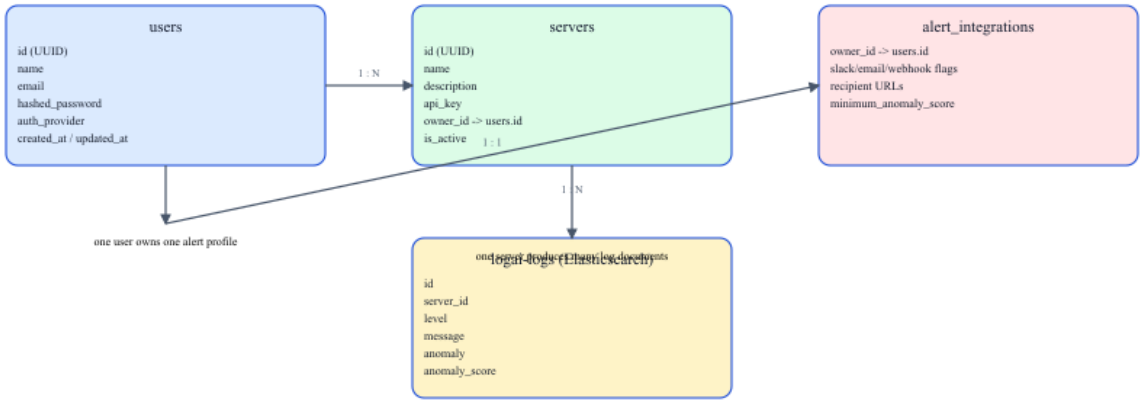

Figure 1.4: Entity Relationship Diagram

1.7 Sequence Diagram

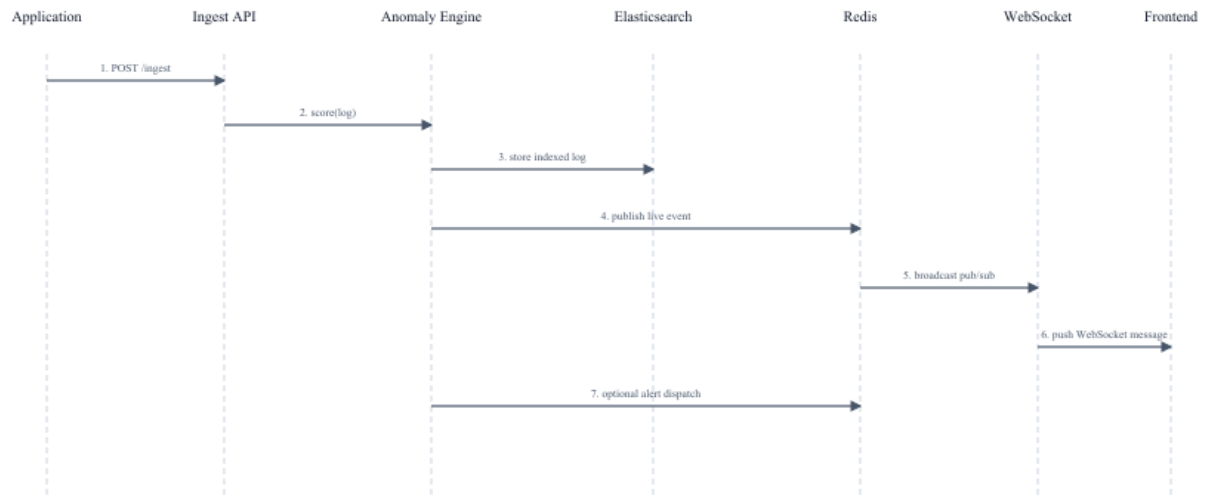

Figure 1.5: Sequence Diagram

1.8 Future Improvements

- Upgrade the current rule-based chat assistant to a stronger retrieval-augmented AI workflow or hosted LLM integration.

- Explore richer anomaly models such as sequence-aware models, ensemble scoring, and feedback-driven threshold tuning.

- Add a hosted deployment path with production-grade monitoring, persistence, TLS, and scaling controls.

- Add organization-level roles, audit trails, retention policies, and multi-tenant support.

- Introduce formal automated E2E browser tests in addition to the current smoke and load scripts.